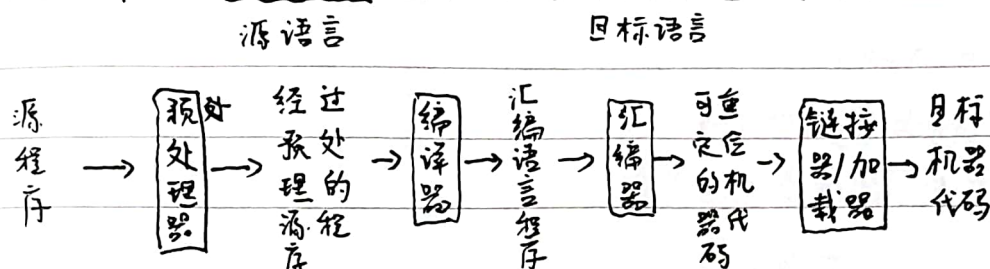


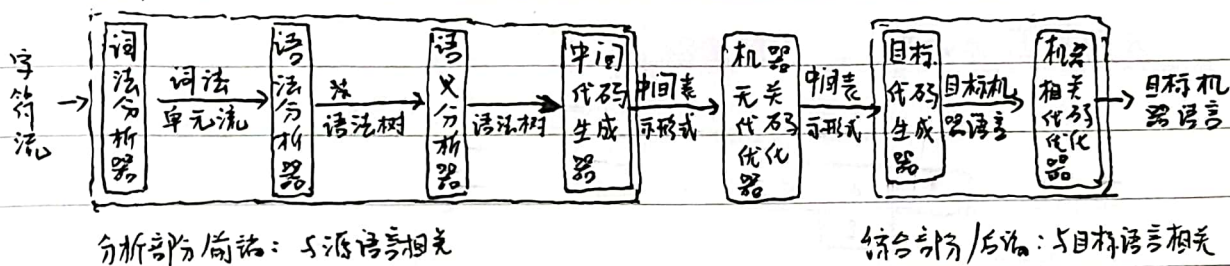
第一章 绪论

1.1 什么是编译

编译：将高级语言翻译成汇编语言或机器语言的过程



1.2 编译系统的结构



• 词法分析主要任务：从左向右逐行扫描源程序的字符，识别出各个单词，确定单词的类型。将识别出的单词转换成统一的[机内表示]——词法单元(token)形式

↳ 关键字：program, if, else, then - 词-码

标识符：变量名、数组名、记录名、过程名 - 多词-码

常量：整型、浮点型、字符型、布尔型 - 型-码

运算符：算术、关系、逻辑 - 词-码或-型-码

界限符：{ } = ; , . - 词-码

• 语法分析器从词法分析器输出的token序列中识别出各类短语，并构造语法分析树。语法分析描述了句子的语法结构。

• 语义分析的主要任务：①收集标识符的属性信息：种属、类型、存储位置、长度、值、作用域、参数和返回值信息

符号表是用于存放标识符的属性信息的数据结构。（符号串表形式）

②语义检查：详见PPT 9个

• 常用的中间表示形式

三地址码 见PPT 10个例子； 语法结构树/语法树



- 三地址指令的表示: 四元式 (op, arg1, arg2, result)
- 三元式 (op, arg1, arg2)

(PPT中有例子)

间接三元式

- 目标代码生成器: 以源程序的[中间表示形式]为输入, 并把它映射到[目标语言]
- 其中一个重要任务是为程序中使用的变量[合理分配寄存器]
- 代码优化: 为改进代码所进行的[等价程序变换], 使其[运行得更快]一些, [占用空间更少]一些, 或者二者兼取

1.3 编译程序的生成

- 编译程序的生成: 1970前 机器语言, 1980后 高级语言

- P: 编译器(程序)

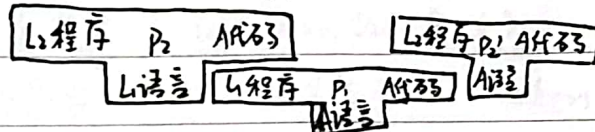
1: 实现语言



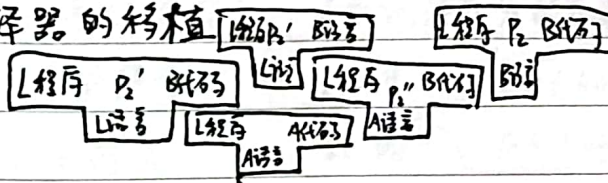
S: 输入的源语言程序

T: 输出的可执行目标程序

- 自展



- 编译器的移植



- 编译器的自动生成: LEX: 词法分析程序生成器

YACC: 语法分析程序生成器

1.4 为什么要学习编译原理

1.5 编译技术的应用

结构化编辑器

智能打印机

静态检测器

文本格式器

数据库查询解释器

高级语言的翻译工具



第二章 语言及其文法

2.1 基本概念

串: 有穷符号的序列

长度 $|s|$ 空串 ϵ x, y 串的连接: xy 空串是单位元 $\epsilon s = s\epsilon = s$

前缀、后缀定义

串 s 的 n 次幂: 将 n 个 s 连接起来

字母表:

eg. 字母表 Σ 是一个[有穷符号集合]字母表 Σ_1 和 Σ_2 的乘积: $\Sigma_1 \Sigma_2 = \{ab \mid a \in \Sigma_1, b \in \Sigma_2\}$ 字母表的 n 次幂: 长度为 n 的符号串构成的集合

字母表的正闭包: 长度正整数的符号串构成的集合

字母表的克林闭包: 任意符号串(长度可以为零)构成的集合

2.2 文法的定义

句子的构成规则

未用尖括号括起来部分表示[语言的基本符号]

eg. $\langle \text{形容词} \rangle \rightarrow \text{title}$

尖括号括起来的部分称为[语法成分]

• 文法的形式化定义: $G = (V_T, V_N, P, S)$ V_T : 终结符集合

基本符号

token

 V_N : 非终结符集合

表示语法成分的非终结符号

语法变量

 P : 产生式集合 $\alpha \rightarrow \beta$ $\alpha \in (V_T \cup V_N)^+$, 且至少含一个 V_N 中元素

头/左部

 S : 开始符号 $S \in V_N$ $\beta \in (V_T \cup V_N)^*$

体/右部

 $V_T \cap V_N = \emptyset$, $V_T \cup V_N$: 文法符号集

• 产生式的简写

 $\alpha \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$ 称 $\beta_1, \beta_2, \dots, \beta_n$ 为 α 的候选式

• 符号约定:

终结符 a, b, c 非终结符 A, B, C 文法符号 X, Y, Z 终结符号串 $u, v, \dots, z$ 文法符号串 α, β, γ

2.3 语言的定义

句子的推导(派生)

— 从生成语言的角度

均根据规则

句子的归约

— 从识别语言的角度

• 若 $s \Rightarrow^* \alpha$, $\alpha \in (V_T \cup V_N)^*$, 则称 α 是 G 的一个句型若 $s \Rightarrow^* w$, $w \in V_T^*$, 则称 w 是 G 的一个句子

· 由文法 G 生成的语言 $L(G) = \{w \mid s \Rightarrow^* w, w \in V_T^*\}$

· 语言上的运算：并，连接，幂，闭包，正闭包，Kleene 闭包

eg. 令 $L = \{A, \dots, Z, a, \dots, z\}$, $D = \{0, 1, \dots, 9\}$. 则 $L(UD)^*$ 表示的是语言的[标识符]

2.4 文法的分类

Chomsky 分法 ~~体系~~ 分类体系

0 型文法： $\alpha \rightarrow \beta$ 无限制文法 / 短语结构文法 $0G/PSG$

$\forall \alpha \rightarrow \beta \in P$, α 中至少包含一个非终结符

1 型文法： $\forall \alpha \rightarrow \beta \in P, |\alpha| \leq |\beta|$ 上下文有关文法 CSG

$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$ ($\beta \neq \epsilon$) $\leftarrow$ 一般形式 (无 ϵ 产生式)

2 型文法： $\forall \alpha \rightarrow \beta \in P, \alpha \in V_N$ 上下文无关文法 CFG

$A \rightarrow \beta$ $\leftarrow$ 一般形式

3 型文法： 正则文法： 右线性文法： $A \rightarrow wB$ 或 $A \rightarrow w$

左线性文法： $A \rightarrow Bw$ 或 $A \rightarrow w$

四种文法之间的关系：逐级限制，逐级包含

2.5 CFG 的语法分析树

· 根结点：文法开始符号

内部结点： $A \rightarrow \beta$ 的左部 A 。子结点标号从左到右构成 β

叶结点：可以是[非终结符]也可以是[终结符]

从左到右排列叶结点得到的符号串称为这棵[树的产出]或[边缘]

· 分析树：推导的图形化表示 (过程中每一个句型 α_i)

· 给定一个句型，其分析树中的每一棵子树的边缘称为该句型的一个

· 如果子树只有父子两代结点，那么这棵子树的

边缘称为该句型的一个[直接短语]

· 如果一个文法可以为某个句子生成多棵分析树，则称这个文法是二义性的

eg. 消歧规则：每个 else 和最近尚未匹配的 if 匹配

· 不存在一个算法，判定二义性。但能给出一些充分条件，满足即无二义性



第三章 词法分析

3.1 单词的描述

正则文法

正则表达式 (一种用来描述正则语言的更紧凑的方法)

正则定义 (Regular Definition)

是具有如下形式的定义序列 $d_1 \rightarrow r_1, d_2 \rightarrow r_2, \dots, d_n \rightarrow r_n$

- 每个 d_i 是一个新符号, 它们都不在字母表 Σ 中, 而且各不相同
- 每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式

eg. PPT C语言标识符、无符号数的正则定义

3.2 单词的识别

3.2.1 有穷自动机 (Finite Automata)

系统只需要根据[当前所处的状态]和[当前面临的输入信息]决定后续行为

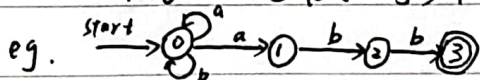
· FA的典型例子: 电梯控制装置

· FA模型: 输入带 

三部分 →

· FA的表示: 转换图

结点: FA的状态. → 初始状态: start箭头指向, 仅一个
终止状态: 双圈表示, 可多个 带标记的有向边

eg. 

· FA定义(接收)的语言 $L(M)$: 如果给定输入串 x , 如果存在一个对应于串 x 的从初始状态到某个终止状态的转换序列, 则称串 x 被该 FA 接收.

· 最长子串匹配原则 (总是选择最长的前缀进行匹配)

3.2.2 FA的分类

确定的 FA (DFA)

非确定的 FA (NFA)

· DFA: 确定的有穷自动机 ~~(DFA)~~ $M = (S, \Sigma, \delta, s_0, F)$
 有穷状态集, 开始状态 (初始状态), $S \subseteq S$
 输入字母表, $S \times \Sigma \rightarrow S$ 的转换函数
 接收(终止)状态集合, $F \subseteq S$

· NFA: 非确定的有穷自动机 $M = (S, \Sigma, \delta, s_0, F)$

将 $S \times \Sigma$ 映射到 2^S 的转换函数

· NFA 和 DFA 是等价的, 它们可以识别相同的语言



· ϵ -NFA: 带有“ ϵ -边”的NFA $M = (S, \Sigma, \delta, s_0, F)$

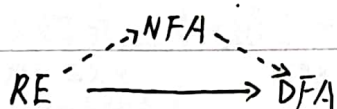
将 $S \times (\Sigma \cup \{\epsilon\})$ 映射到 2^S 的转换函数

$\forall s \in S, a \in \Sigma \cup \{\epsilon\}$, $\delta(s, a)$ 表示从状态 s 出发, 沿着标记 a 边所能到达的状态集合

· ϵ -NFA 和 NFA 也是等价的

· DFA 算法的实现 详见 PPT P24

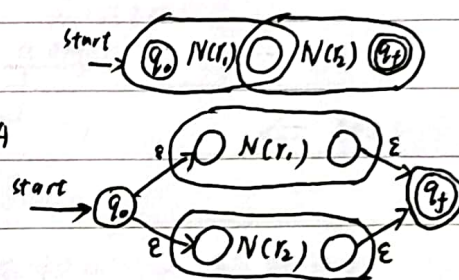
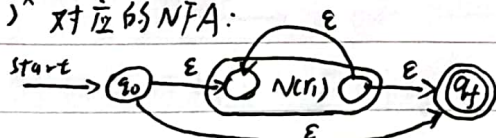
3.2.3 从正则表达式到有限自动机



· $RE \rightarrow NFA$: ① $r = r_1 r_2$ 对应的 NFA

② $r = r_1 \mid r_2$ 对应的 NFA

③ $r = (r_1)^*$ 对应的 NFA:



· 从 ϵ -NFA 到 DFA 的转换: 子集构造法

ϵ -closure(s): 能从 NFA 的状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合

ϵ -closure(T): 能从 T 中某个状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合, $\cap U_{s \in T} \epsilon$ -closure(s)

$\rightarrow$ 注意: 不含 $\epsilon a, a \epsilon$
move(T, a): 能从 T 中某个状态 s 出发通过标记为 a 的转换到达 NFA 状态的集合

ϵ -closure(T) 的计算: 维护一个栈, 从栈顶元素出发找新元素入栈 (详见 PPT P32)

子集构造法思路: 维护一个 $Dstate$ 集合为 DFA 的状态, 初始均未标记, 每标记一个状态便将该状态相关的所有的新状态 $Dstate$ 加入集合, 并建立对应转换函数。 (详见 PPT P31)

3.2.4 识别单词的 DFA

· eg. 识别标识符 | 无符号数 | 带进制无符号数 | 注释 | token 的 DFA (详见 PPT)

3.3 词法分析阶段的错误处理

· 词法错误的类型

单词拼写错误 非法字符

· 词法错误检测: 如果当前状态与当前输入符号在转换表对应项中的信息为空, 而当前状态又不是 [终止状态], 则调用 [错误处理程序]



· 错误处理：查找已扫描字符串中最后一个对应于某终态的字符

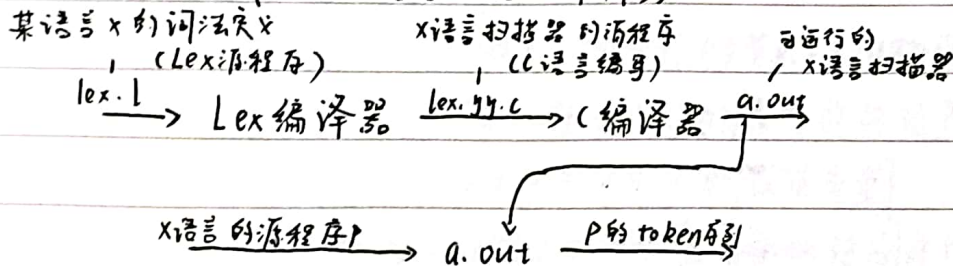
找到了：退回该位置，把状态回到初始状态找下一个单词

没找到：确定出错，采用错误恢复策略

· 错误恢复策略：eg. “恐慌模式”恢复：从剩余的输入中不断删除字符，直到词法分析器能够在剩余输入的头发现一个正确的字符为止。

3.4 词法分析生成工具 Lex

Lex 的构成：Lex 语言 + Lex 编译器



Lex 的程序结构：声明部分 + 转换规则 + 辅助过程 (详见 PPT P46)

扫描器自动生成的意义：

· Lex 的构成加快了分析器的实现速度 (程序员只需在很高层次的模式上描述软件，就可以依赖自动生成工具来生成详细的代码)

· 修改扫描器的工作变得更加简单 (无需修改那些受到影响的模式，无需改写整个程序)



第四章 语法分析

引言 · 主要任务: 根据给定的[文法], 识别[输入句子]的各个成分, 从而构造出句子的[分析树]

- 大部分程序设计语言的[语法构造]可以用[CFG]来描述, 以[Token]为[终结符]
- 大部分语法分析器都期望文法是[无二义性]的.

语法分析的种类: 自顶向下分析、自底向上分析

4.1 自顶向下的分析

- 从分析树的顶部(根节点)向底部(叶节点)方向构造分析树
- 最左推导: 总是选择每个句型的[最左非终结符]进行替换
如果 $s \Rightarrow_m^* \alpha$, 则称 α 是当前文法的[最左句型]
- 最右推导: 总是选择每个句型的[最右非终结符]进行替换
如果 $s \Rightarrow_m^* \alpha$, 则称 α 是当前文法的[最右句型]
- 在自底向上的分析中, 总是采用[最左归约]的方式, 因此把最左归约称为[规范归约], 而[最右推导]相应地称为[规范推导]
- 最左推导和最右推导具有唯一性

自顶向下语法分析的通用形式 — 递归下降分析

由一组[过程]组成, 每个过程对应文法的一个[非终结符]

从文法[开始符号] S 对应的过程开始, 其中[递归调用]文法中其它[非终结符]对应的过程. 如果 S 对应的过程恰好扫描了[整个输入串]则成功完成语法分析

暂时存在问题 1: 同一非终结符的多个候选式存在共同前缀, 将导致[回溯]现象
可解决 < 问题 2: 左递归文法会使递归下降分析器陷入[无限循环]

↳ 由直接左递归 $A \rightarrow A\alpha$, 间接左递归产生.

↳ 消除直接左递归的一般形式 (转换成右递归)

$$A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_n | \beta_1 | \beta_2 | \dots | \beta_m \quad (\alpha_i \neq \epsilon, \beta_j \text{ 不以 } A \text{ 开头})$$

$$\downarrow$$

$$A \rightarrow \beta_1 A' | \beta_2 A' | \dots | \beta_m A'$$

$$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \dots | \alpha_n A' | \epsilon$$

消除左递归是要付出代价的 — 引进了一些非终结符和 ϵ -产生式



· 消除间接左递归 eg. $S \rightarrow Aa|b$ 有 $S \Rightarrow Aa \Rightarrow Sda$

$A \rightarrow Ac|Sd|\epsilon$

解决: 将 S 的定义代入 A -产生式, 消除 A 产生式的直接左递归.

· 消除左递归算法: 按 $A_1, A_2, \dots, A_n$ 排序, 使 A_i 中不出现 $A_j (i > j)$. 消除 A_i 左递归 (详见 PPT P19)

优化: · 提取左公因子 目的: 通过改写产生式来推迟决定, 当读入了足够多的输入, 获得足够信息后再做出正确的选择

· 预测分析: 是 [递归下降分析] 技术的一个特例, 通过在输入中向前看 [固定个数 (通常是一个) 符号] 来选择正确的 A -产生式. (向前看 k 个为 $LL(k)$ 文法类)

· 预测分析 [不需要回溯], 是一种 [确定的] 自顶向下分析方法

4.2 预测分析法

4.2.1 $LL(1)$ 文法

· S -文法 简单的确定性文法 (S -文法不含 ϵ 产生式)

→ 每个产生式的右部都以 [终结符] 开始

→ 同一非终结符的各个候选式的 [首终结符] 都不同

· 非终结符 A 的后继符号集 $FOLLOW(A) = \{a | S \Rightarrow^* aA\alpha, a \in V_T, \alpha \in (V \cup V^*)^*\}$

· 产生式 $A \rightarrow \beta$ 的可选集: 选用该产生式进行推导时对应的输入符号的集合, 记为 $SELECT(A \rightarrow \beta)$

· q -文法 → 每个产生式右部或为 ϵ , 或以终结符开始

→ 具有相同左部的产生式有不相交的可选集

q -文法不含右部以非终结符打头的产生式

· 串首符号集: 串首第一个符号, 并且是终结符. 简称首终结符

① 对于 $\forall \alpha \in (V \cup V^*)^+$, $FIRST(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in V_T, \beta \in (V \cup V^*)^*\}$

② 如果 $\alpha \Rightarrow^* \epsilon$, 那么 $\epsilon \in FIRST(\alpha)$

· 求 $FIRST(X)$ 的算法: ① 若 X 是一个终结符, 那么 $FIRST(X) = \{X\}$

② 若 X 是一个非终结符, 且 $X \rightarrow Y_1 \dots Y_k \in P (k \geq 1)$, 那么如果对于某个 i , a 在 $FIRST(Y_i)$ 中且 ϵ 在所有 $FIRST(Y_1) \dots FIRST(Y_{i-1})$ 中, (即 $Y_1 \dots Y_{i-1} \Rightarrow^* \epsilon$), 那就把 a 加入 $FIRST(X)$ 中. 若 $Y_1 \dots Y_k \Rightarrow^* \epsilon$, 那么将 ϵ 加入 $FIRST(X)$ 中

③ 如果 $X \rightarrow \epsilon \in P$, 那么将 ϵ 加入 $FIRST(X)$ 中.



• 产生式 $A \rightarrow \alpha$ 的可选集 SELECT

若 $\epsilon \notin \text{FIRST}(\alpha)$, 那么 $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$

若 $\epsilon \in \text{FIRST}(\alpha)$, 那么 $\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) - \{\epsilon\}) \cup \text{FOLLOW}(A)$

• 文法 G 是 LL(1) 的, 当且仅当 G 的任意两个具有相同左部的产生式

$A \rightarrow \alpha \mid \beta$ 满足下面的条件:

① 不存在终结符 a 使得 α 和 β 都能够推导出以 a 开头的串

② α 和 β 至多有一个能推导出 ϵ

③ 若 $\beta \Rightarrow^* \epsilon$, 则 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

若 $\alpha \Rightarrow^* \epsilon$, 则 $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

• LL(1) 中第一个 L 表示从左向右扫描输入, 第二个 L 表示产生最左推导

"1" 表示在每一步中只需要向前看一个输入符号来决定语法分析动作。

• FOLLOW(A) 计算方法:

① 将 ϵ 放入 FOLLOW(S) 中, 其中 S 是开始符号, ϵ 是输入右端的[结束标记]

② 如果存在一个产生式 $A \rightarrow \alpha B \beta$, 那么 $\text{FIRST}(\beta)$ 中除 ϵ 之外所有符号都在 FOLLOW(B) 中

③ 如果存在一个产生式 $A \rightarrow \alpha B$, 或存在产生式 $A \rightarrow \alpha B \beta$ 且 $\text{FIRST}(\beta)$ 包含 ϵ , 那么 FOLLOW(A) 中的所有符号都在 FOLLOW(B) 中。

• 预测分析表 详见 PPT P41 形式

• 如何实现预测分析

递归的方式: 基于^{分析}预测表对[递归下降分析法]进行扩展

非递归的方式: [显式地]维护一个[栈结构]来模拟[最左推导]过程

4.2.2 递归的预测分析法

主程序: program DESCENT

begin GETNEXT(TOKEN)

PROGRAM(TOKEN)

if TOKEN $\neq$ '\$' then ERROR

end.

A(TOKEN) 注: $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$

{ if Token $\in$ SELECT($A \rightarrow \alpha_1$)

{ for $j=1$ to n
if $X_j \in V_T$ if $X_j = \text{Token}$ GetNextToken

else Error()

} else $X_j \in V_N$ $X_j(\text{Token})$ }

if Token $\in$ SELECT($A \rightarrow \alpha_2$)

}



4.2.3 非递归的预测分析法

· 非递归的预测分析显式地维护一个栈，而不是通过递归调用的方式隐式地维护栈。这样语法分析器可以模拟最左推导过程

详见PPT P55 例子 (栈中是部分未匹配的句型)

· 表驱动的预测分析法 详见PPT P56 if else 共4部分

	递归的预测分析法	非递归的预测分析法
程序规模	程序规模较大	主控程序规模较小
	不需载入分析表	需载入分析表(表较小)
直观性	较好	较差
效率	较低	分析时间大约正比于待分析程序长度
自动生成	较难	较易

· 预测分析法实现步骤

1) 构造文法

2) 改造文法: 消除二义性, 消除左递归, 消除回溯

3) 求每个变量的FIRST集和 FOLLOW集, 从而求得每个候选式 SELECT集

4) 检查是不是 LL(1) 文法。若是, 构造预测分析表

5) 对于递归的预测分析, 根据预测分析表为每一个非终结符编写一个过程; 对于非递归的预测分析, 实现表驱动的预测分析法

· 无二义性 vs. 确定性 $\text{无二义性} \Rightarrow \text{确定性 (无二义性的充分条件)}$

4.2.4 预测分析中的错误检测

· 两种情况下可以检测到错误 ① 栈顶终结符和当前输入符号不匹配

② 栈顶非终结符与当前输入符号在预测分析表对应项中的信息为空

· 预测分析中的错误恢复

恐慌模式

eg. ① $M[A, a]$ 为空 弹出, 忽略输入符号 a

② $M[A, a]$ 是 synch , 弹出非终结符 A

③ 栈顶终结符和输入符号不匹配, 则弹出栈顶的终结符.



4.4.1 LR(0) 分析

- LR(0) 项目: $A \rightarrow d_1 \cdot d_2$
- 增广文法: 加上新开始符号 S' 和产生式 $S' \rightarrow S$ 而得到文法
 - 作用: 使分析器只有一个接受状态.
- 初始项目、归约项目、接收项目
- 后继项目、项目闭包集、活前缀
- 计算给定项目集 I 的闭包 $CLOSURE(I) = I \cup \{B \rightarrow \gamma \mid A \rightarrow \alpha \cdot B \beta \in I, B \rightarrow \gamma \in P\}$
- 返回项目集 I 对应于文法符号 X 的后继项目集闭包

$$GOTO(I, X) = CLOSURE(\{A \rightarrow \alpha X \beta \mid A \rightarrow \alpha \cdot X \beta \in I\})$$
- 规范 LR(0) 项集族 $C = \{I_0\} \cup \{I_j \mid \exists j \in C, X \in V \cup \$, I_j = GOTO(I, X)\}$
- $CLOSURE(\{S' \rightarrow S\}) \rightarrow C$ C : 自动机状态集合
- for each $I_i \in C$ 注 $GOTO()$ 和 $GOTO[]$ 不同
 - if $A \rightarrow \alpha \cdot a \beta \in I_i$: $\exists I_j = GOTO(I_i, a)$; $ACTION[I_i, a] = S_j$
 - if $A \rightarrow \alpha \cdot B \beta \in I_i$: $\exists I_j = GOTO(I_i, B)$; $GOTO[I_i, B] = j$
 - if $A \rightarrow \alpha \cdot \epsilon \in I_i$: $\begin{cases} \text{if } A = S' (S' \rightarrow \$ \in I_i) ACTION[I_i, \$] = acc \\ \text{if } A \neq S' \text{ for } \forall a \in V \cup \{ \$ \} \text{ do } ACTION[I_i, a] = \text{reject} \end{cases}$ $\downarrow$ 产生式的编号
- 如果 LR(0) 分析表中没有语法分析动作冲突, 那么给定的文法就称为 LR(0) 文法.
- 不是所有 CFG 都能用 LR(0) 方法, 也就是说 CFG 不总是 LR(0) 文法.
- LR(0) 自动机为什么消解不了冲突: 考虑了上文, 但未考虑下文

4.4.2 SLR 分析

- 设有 m 个移进项目, n 个归约项目. 若 $\{a_1, \dots, a_m\} \cap FOLLOW(B_1) \dots FOLLOW(B_n)$ 两两不相交, 则可以根据下一个输入符号决定移进 or 归约 or 报错
- eg. 表达式文法的 SLR 分析表. 见 PPT P6
- SLR 分析表构造算法: 将 LR(0) 中 $A \neq S'$ 中的 $\forall a \in V \cup \{ \$ \}$ 改为 $\forall a \in FOLLOW(A)$
- 如果给定 SLR 分析表中不存在有冲突的动作, 那么该文法称为 SLR 文法.



4.4.3 LR(1) 分析

- LR(1) 分析法的提出: 对于产生式 $A \rightarrow \alpha$ 的归约, 在不同的使用位置, A 会要求不同的后继符号。在特定的位置, A 的后继符是集合 $FOLLOW(A)$ 的子集。
- 规范 LR(1) 项目 $[A \rightarrow \alpha \cdot \beta, a]$, $a \in FOLLOW(\beta)$ 为展望符。
- 等价的 LR(1) 项目 $[A \rightarrow \alpha \cdot \beta, a] \xleftrightarrow{B \rightarrow \gamma \in P} [B \rightarrow \cdot \gamma, b]$, $b \in FIRST(\beta a)$
- 除展望符外, 两个 LR(1) 项目集是相同的, 则称这两个 LR(1) 项目集是同心的
- $CLOSE(I) = I \cup \{ [B \rightarrow \cdot \gamma, b] \mid [A \rightarrow \alpha \cdot \beta, a] \in I, B \rightarrow \gamma \in P, b \in FIRST(\beta a) \}$
- $GOTO(I, X) = CLOSE(\{ [A \rightarrow \alpha X \cdot \beta, a] \mid [A \rightarrow \alpha \cdot X \beta, a] \in I \})$
- 为文法 G 构造 LR(1) 项集族 详见 PPT P22
- LR 分析表构造算法 将 $A \neq S'$ 情况改为:
 - 详见 PPT P23 $\text{if } [A \rightarrow \alpha \cdot, a] \in I \text{ 且 } A \neq S' \text{ then ACTION}(i, a) = r_j$
- 各种 LR 分析表构造方法不同之处在于归约项目的处理上。

4.4.4 LALR 分析

- 基本思想: 寻找具有相同核心的 LR(1) 项集, 并将这些项集合并为一个项集。然后根据合并后得到的项集族构造语法分析表。
- 合并同心项集不会产生进一步归约冲突, 但可能产生归约-归约冲突
 ↓
 但可能会推迟错误的发现
- 形式上与 LR(1) 相同, 大小上与 LR(0) / SLR 相当
- 分析能力介于 SLR 与 LR(1) 两者之间 $LR(0) < SLR < LALR(1) < LR(1)$
- 合并后的展望符集合仍为 FOLLOW 集的子集。

4.4.5 二义性文法的 LR 分析

- 每个二义性文法都不是 LR 的
- eg. 二义性算术表达式文法 利用[优先级]和[结合性]解决冲突
- 二义性 if 语句文法的 LR 分析 优先选择移进 else

4.4.6 LR 分析中的错误处理

- 语法错误的检测
- 错误恢复策略: 恐慌模式错误恢复、短语层次错误恢复



第五章 语法指导翻译

语法分析
 语义分析
 中间代码生成

} 语⋈翻译

} 语法指导翻译
 CFG

语法指导翻译使用CFG来引导对语言的翻译,是一种[面向文法]的翻译技术

- 如何表示语义信息: 为CFG中的[文法符号]设置[语义属性], 用来表示语法成分对应的[语义信息]
- 如何计算语义属性: 文法符号的语义属性值是用文法符号所在产生式(语法规则)相关联的[语义规则]来计算的
- 语法指导定义(SDD): 是对CFG的推广: ①将每个[文法符号]和一个[语义属性]集合相关联 ②将每个[产生式]和一组[语义规则]相关联
- 语法指导翻译方案(SDT): SDT是在产生式右部嵌入了程序片段的CFG, 这些程序片段称为[语义动作]。按照惯例, 语义动作放在花括号内。

5.1 语法指导定义 SDD (定义同上)

· 文法符号的属性: 综合属性、继承属性

非终结符: 子结点本身 父结点兄弟结点本身

终结符: 从词法分析器又获得[综合属性值]

- 注释分析树: 每个节点都带有属性值的分析树
- 属性文法: 一个没有副作用的SDD有时也称为属性文法
属性文法的规则仅仅通过其它属性值和常量来定义一个属性值
- SDD的求值顺序: 依据属性之间的依赖关系
- 依赖图: 描述了分析树中结点属性间依赖关系的有向图
如果图中没有环, 那么至少存在一个拓扑排序

5.2 S-属性定义与L-属性定义

- 仅仅使用[综合属性]的SDD称为[S属性的SDD]或[S-属性定义]、S-SDD
S-属性定义可以在自底向上的语法分析过程中实现
- L-属性定义(L属性的SDD或L-SDD): 依赖图的不能从右到左



· L-SDD的正式定义: 一个SDD是L-SDD, 当且仅当它的每个属性要么是一个综合属性, 要么是如下条件的继承属性: 假设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$ 其右部符号 $X_i (1 \leq i \leq n)$ 的继承属性仅依赖于下列属性: ① A的继承属性 ② 产生式中 X_i 左边的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性. ③ X_i 本身的属性, 但 X_i 的全部属性不能在依赖图中形成环路.

· 虚属性: eg. print ..., addtype(...)

5.3 语法制导翻译方案 SDT

· 语法制导翻译方案(SDT)是在产生式右部中嵌入了程序片段(语义动作)的CFG

· SDT可以看作是SDD的具体实施方案

怎么算+何时算

怎么算

① 将 S-SDD 转换为 SDT

· 将一个S-SDD转换为SDT的方法: 将每个语义动作都放在产生式的最后

· 如果一个S-SDD的基本文法可以使用LR分析技术, 那么它的SDT可实现

· 语法分析器的扩展: 为每个栈记录增加[属性值字段], 存放文法符号的[综合属性值]。在每次[归约]时[调用]计算综合属性值的[语义子程序]。

· 若支持多个属性: 法1: 使栈记录变得足够大. 法2: 在栈记录中保存指针.

② 将 L-SDD 转换为 SDT

· 将计算一个产生式左部符号的[综合属性]的动作放置在这个产生式右部的[最右末端]; 将计算某个非终结符号A的[继承属性]的动作插入到产生式右部中[紧靠在A的本次出现之前]的位置上。

· 如果一个L-SDD的基本文法可以使用LL分析技术, 那么它的SDT可以在LL或LR语法分析过程中实现。

5.4 L-SDD 的自顶向下翻译 (在预测分析的同时实现语义翻译)

5.4.1 在非递归的预测分析过程中进行翻译

扩展语法分析栈

action	A	A _{syn}	Symbol
			Value

指向被执行的语义动作代码的指针 A的继承属性 A的综合属性



· 分析栈中的每一个记录都对应着一段执行代码

综合记录[出栈], 要将[综合属性值]复制给后面特定语义动作

变量展开时(即变量本身[出栈]时)如果其含有继承属性, 则要将[继承属性值]复制给后面的特定的语义动作。

5.4.2 在递归的预测分析过程中进行翻译

算法: · 为每个[非终结符 A]构造一个[函数], A 的每个[继承属性]对应函数的一个[形参], 函数的[返回值]是 A 的[综合属性值]。对出现在 A 产生式中的每个文法符号的每个[属性]都设置一个[局部变量]

· [非终结符 A]的代码根据当前的输入决定使用哪个产生式

5.5 L-属性定义的自底向上翻译

给定一个以 LL 文法为基础的 L-属性定义, 可以修改这个文法, 并在 LR 语法分析过程中计算这个新文法上的 SDD

· 首先构造 SDT, 在每个[非终结符之前]放置语义动作来计算它的[继承属性], 并在[产生式后端]放置语义动作计算[综合属性]

· 对于每个内嵌[语义动作], 向文法中引入一个[标记非终结符]来替换它, 每个这样的位置都有一个[不同的标记], 并且对于任意一个标记 M 都有一个产生式 $M \rightarrow \epsilon$

· 如果标记非终结符 M 在某个产生式中 $A \rightarrow \alpha_1 a \alpha_2 \beta$ 中替换了语义动作 a , 对 a 进行修改得到 a' , 并且将 a' 关联到 $M \rightarrow \epsilon$ 上。动作 a' 复制

(a) 将动作 a 需要的 A 或 a 中符号的任何属性作为 M 的[继承属性]进行

(b) 按照 a 中方法计算各个属性, 但是将计算得到这些属性作为 M 的[综合属性]



第六章 中间代码生成

6.1 声明语句的翻译

- 声明语句翻译的主要任务：收集标识符的[类型]等属性信息，并为每一个名字分配一个[相对地址]

名字的[类型]和[相对地址]信息保存在相应的[符号表]记录中

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将[类型构造符]作用于[类型表达式]可以构成新的类型表达式
数组构造符 array 指针构造符 pointer 枚举构造符 X
结构构造符 $\rightarrow$ 记录构造符 record

- 局部变量的存储分配 `enter (name, type, offset)`

从[类型表达式]可以知道该类型在[运行时刻]所需的存储单元数量称为[类型的宽度] (width)

在[编译时刻]，可以使用[类型的宽度]为每一个名字分配一个[相对地址]

6.2 赋值语句的翻译

6.2.1 简单赋值语句的翻译

- 赋值语句的基本文法 详见 PPT P18

- 赋值语句翻译的主要任务：生成对表达式求值的三地址码

`newtemp()`: 生成一个新的临时变量 `t`，返回 `t` 的地址

`gen(code)`: 生成三地址指令 `code`

`lookup(name)`: 查询符号表，返回 `name` 对应的记录

- 增量翻译 (Incremental Translation): 在增量方法中，`gen()`不仅要构造出一个新的三地址指令，还要将它添加到至今为止已生成的指令序列之后

6.2.2 数组引用的翻译

- 将[数组引用]翻译成[三地址码]时要解决的主要问题是确定[数组元素的存放地址]，也就是数组元素的寻址



· 数组元素寻址

- 维数组 $\text{base} + i \times w$
 ↓ ↓
 基地址 偏移地址

= 二维数组 $\text{base} + i_1 \times w_1 + i_2 \times w_2$

k 维数组 $\text{base} + i_1 \times w_1 + \dots + i_k \times w_k$

eg. `type(a) = array(3, array(5, int))`

`addr(a[i1][i2]) = base + i1 * 20 + i2 * 4`

· 数组元素寻址 SDT L 的综合属性 L.type, L.offset, L.array

eg. `array(8, int) ←` 数组元素的类型 w_j 数组名在符号表的入口地址

· 符号表的组织 — 数组

 基本属性
 名字 种属 类型 地址 扩展属性指针 扩展属性

6.3 控制语句的翻译

· 控制流语句的基本文法 详见 PPT P41

· 控制流语句的代码结构

eg. `S → if B then S1 else S2.`

布尔表达式 B 被翻译成由[跳转指令]构成的跳转代码

继承属性: B.true, B.false, S.next 用指令的标号标识一条三地址指令

· `newlabel(l)`: 生成一个用于存放标号的新的临时变量 L, 返回变量地址

`label(l)`: 将下一条三地址指令的标号存放到底址 L 中

· 控制流语句 SDT 编写要点

· 分析每一个非终结符之前: 先计算[继承属性], 再仔细观察代码结构图中该非终结符对应方框顶部是否有[导入箭头]。如果有, 调用 `tbl label` 函数

· 上一个代码框执行完[不顺序]执行下一个代码框, 生成一条[显式跳转指令]

· 有[自下而上的箭头]时, 设置[begin 属性]。且定义后直接调用 `label(l)` 函数绑定地址

· 布尔表达式的翻译 布尔表达式的基本文法 详见 PPT P51

· 在跳转代码中, 逻辑运算符 &&, || 和 ! 被翻译成[跳转指令]。运算符本身不出现在代码中, 布尔表达式的值是通过代码序列中位置来表示的。

· 基础文法: 可以使用 LR 分析技术



· SDD: L-SDD

赋值语句: 只定义了[综合属性]

分支、循环语句: 只定义了[继承属性], 且不依赖右兄弟节点属性值

· SDT的通用实现方法: 首先[建立]一棵语法[分析树]

然后按照[从左到右]的[深度优先]顺序来[执行]这些[动作]

if-then 语句, $B \rightarrow E_1 \text{ or } E_2$ 语句, $B \rightarrow B_1 \text{ or } B_2$ 语句, $B \rightarrow B_1 \text{ and } B_2$ 语句的修改 详见 PPT P67-70

6.4 回填

基本思路: 生成一个跳转, 暂时不指定该跳转指令的[目标标号]。这样的指令都被放入由跳转指令组成的[列表]中。[同一个列表中的所有跳转指令具有相同的目标标号]。等到能够确定正确的目标标号, 才去填充这些指令的目标标号。

· 非终结符 B 的综合属性 $B.truelist, B.falselist$ 包含跳转指令的列表

$makelist(i)$: 创建一个只包含 i 的列表, i 是跳转指令的标号, 函数返回指向创建的新列表的指针

$merge(p_1, p_2)$: 将 p_1 和 p_2 指向的列表进行合并, 返回指向合并后的列表的指针

$backpatch(p, i)$: 将 i 作为目标标号插入到 p 所指向列表中的各指令中

$nextquad$: 即将生成的下一条指令的标号。

布尔表达式的回填 详见 PPT P78-88

· 控制流语句的回填 $S.nextlist$ 综合属性

· 回填技术 SDT 编写要点

文法改造: 在 list[箭头指向的位置]设置[标记非终结符 M]

在产生式末尾的语义动作中: 计算[综合属性], 调用 $backpatch()$ 函数[回填]各个 list, 生成必要的附加指令。

6.5 switch 语句的翻译

· switch 语句的两种翻译 详见 PPT P100-101

· 在代码生成阶段, 根据分支的个数以及这些值是否在一个[较小的范围内], 这种条件跳转指令序列可以被翻译成[最高效的 n 路分支]



· 指令 $\text{case } V_i \text{ } L_i$ 和 $\text{if } t = V_i \text{ goto } L_i$ 的含义相同, 但是 case 指令[更加容易]被最终的代码生成器[探测]到, 从而对这些指令进行[特殊处理].

6.6 过程调用语句的翻译

· 需要一个队列 q 存放 $E_1.\text{addr}, E_2.\text{addr}, \dots, E_n.\text{addr}$.

$S \rightarrow \text{call id}(Elist) \{ n=0; \text{for } q \text{ 中每个 } t \text{ do } \{ \text{genl'param' } t \}; n=n+1; \} \text{genl'call'id.addr; } n \}$

$Elist \rightarrow E \{ \text{将 } q \text{ 初始化为只包含 } E.\text{addr} \}$

$Elist \rightarrow Elist, E \{ \text{将 } E.\text{addr} \text{ 添加到 } q \text{ 的队尾} \}$



- 顺序分配法: 按照过程出现的先后顺序逐段分配存储空间
各过程的活动记录占用互不相交的存储空间

优点: 处理上简单 缺点: 对内存空间的使用不够经济合理

- 层次分配法: 通过对过程间调用关系进行分析, 凡属无相互调用关系的并列过程, 尽量使其局部数据[共享]存储空间。

$B[n][n]$ 过程调用关系矩阵

$Units[n]$: 过程所需内存量矩阵

$base[i]$, $allocated[i]$ 及代码例子详见 PPT P15-16

7.3 栈式存储分配

- 当一个过程被[调用]时, 该过程的活动记录被[压入]栈; 当过程结束时, 该活动记录被[弹出]栈

· 这种安排不仅允许活跃时段不交叠的多个过程调用之间[共享空间], 而且允许以如下方式为一个过程编译代码: 它的非局部变量的[相对地址总是固定的], 和过程调用序列无关。

- 活动树: 用来描述程序[运行]期间[控制]进入和离开各个活动情况的树

- 树中每一个[结点]对应于一个[活动]。根节点是启动执行程序 main 过程的活动

- 在表示过程 p 的某个活动的结点上, 其[子结点]对应于被 p 的这次活动[调用]的各个过程的活动。按照这些[活动被调用的顺序], 自左向右地显示它们。一个子结点必须在其[右兄弟]结点的活动[开始]之前[结束]。

- 每一个[活跃的活动]都有一个位于[控制栈]中的[活动记录]。

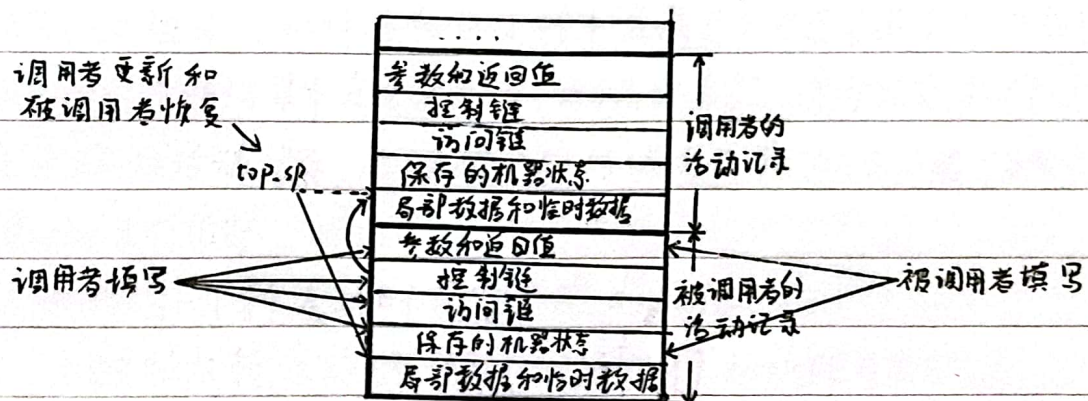
- 根在栈底; 当前活动记录在栈顶。

- 栈中[全部活动记录的序列]对应于活动树中[到达控制所在活动结点的路径]。

· 设计活动记录的一些原则: ①在[调用者和被调用者]之间传递的值一般被放在[被调用者的活动记录开始的位置], 这样它们可以尽可能地靠近调用者的活动记录。②固定长度的项被放置在中间位置(如控制链、访问链和机器状态字)③在[早期不知道大小的项]被放在活动记录的[尾部]④栈顶指针寄存器 top-sp 指向活动记录中[局部数据的位置], 以该位置作为基地址。
开始



- 调用序列: 实现[过程调用]的代码段。为一个活动记录在栈中分配空间, 并在此记录的字段中填写信息。
- 返回序列: 实现[过程返回]的代码段。恢复机器状态, 使得调用过程能够在调用结束后继续执行。
- 一个调用代码序列中的代码通常被分割到调用过程(调用者)和被调用过程(被调用者)中。返回序列也是如此。
- 调用者和被调用者之间的任务划分



- 变长数据的分配: 在[现代程序设计语言]中, [在编译时刻不能确定大小的对象]将被分配在堆区。但是如果它们是[过程的局部对象], 也可以将它们分配在[运行时刻栈]中。尽量将对象放置在栈区的原因: 可以[避免]对它们的空进行[垃圾回收], 也就[减少]了相应的[开销]。
- 只有一个数据对象[局部于某个过程], 且[在此过程结束时它变得不可访问], 才可以使用[栈]为这个对象分配空间。

7.4 非局部数据的访问

- 一个过程除了可以使用[过程自身]声明的[局部数据]以外, 还可以使用[过程外]声明的[非局部数据]。eg. 全局数据, [外围过程]定义的数据

如何访问非局部数据 访问链(静态链)
display表(嵌套层次显示表)

· 访问链

静态作用域规则: 只要过程b的声明嵌套在过程a的声明中, 过程b就可以访问过程a中声明的对象。



在相互嵌套的过程的活动记录之间建立一种称为[访问链]的指针，使得[内嵌]的过程可以访问[外层]过程中声明的对象。

建立访问链属于[调用序列]的一部分

外层调用内层，本层调用本层，内层调用外层（三种 $x \rightarrow y$ 情况）

$n_y = n_x + 1$, y 指向 x

直接复制

沿访问链走 $n_x - n_y + 1$

嵌套深度是在编译阶段通过[静态分析]就能确定的。

display表（嵌套层次显示表）

访问链访问外层过程名效率低

指针数组 d ， $d[i]$ 指向运行栈中最新建立的嵌套深度为 i 的过程的活动记录。如果要访问某个嵌套深度为 i 的非局部名字 x ，只要沿着指针 $d[i]$ 找到 x 所属过程的活动记录，再根据已知的偏移量就可以在活动记录中找到 x 。

display表的维护

每[开始]一个新活动时都要[修改] display表

当控制从新活动[返回]时，必须[恢复] display表的状态

当过程 p 被[调用]，除了要修改 $d[n_p]$ 还要[保存] $d[n_p]$ [原来的值]

7.5 参数传递

形式参数：在过程[定义]中使用的参数

实际参数：在[调用]过程中使用的参数

形参和实参相关联的几种方法：传值、传地址、传值结果、传名

7.5.1 传值 (call-by-value) eg. Pascal 的值参数，C/C++ 的缺省参数传递

7.5.2 传地址 (call-by-reference) eg. Pascal 的 Var 参数，C/C++ 的传引用

7.5.3 传值结果 (call-by-value-result) eg. Fortran

每个形参对应一个[地址]一个[值]单元，过程体中使用值

过程完成返回前会把值[返回]地址^址对应的单元

7.5.4 传名 (call-by-name) eg. ALGOL 60

相当于把[被调用过程]的过程体[抄到]调用出现的地方，但把其中出现的[形参]都替换成相应的[实参]

传名方式中存在重命名问题



7.6 符号表

· 符号表是用于存放[标识符]的[属性信息]的数据结构
种属、类型、存储位置、长度、作用域、参数和返回值信息

· 符号表的作用：辅助代码生成，一致性检查

· 主要操作：
[声明]语句的翻译(定义性出现) 填、查
可执行语句的翻译(使用性出现) 查

· 单个过程符号表的组织

方法一：一张大表

问题：不同种属的名字所需存放的属性信息在数量上的差异会造成符号表空间的浪费。

方法二：多张子表(按种属分)

问题：为避免重名问题，插入或查找某个符号时需要查看所有符号表，从而造成时间上的浪费。

解决办法：基本属性(直接存放在符号表中) + 扩展属性(动态申请内存)

· 多个过程符号表的组织

两个问题：①刻画过程之间的[嵌套]关系(作用域信息) ②重名问题

常用组织方式：每个[过程]建立一个符号表，同时需要建立起这些符号表之间的联系，用来刻画[过程]之间的[嵌套关系]。

· 嵌套过程声明语句的文法

$P \rightarrow D$ $D \rightarrow D \mid \text{proc id}; D \mid \text{id}: T;$

mktable (previous) addwidth (table, width) enterproc (table, name, newtable)

enter (table, name, type, offset) tblptr offset nil header

· 标识符的基本处理方法

声明语句中：查[本层]符号表，判断是否[重复声明]，否则加入[新登记项]

可执行语句中：先查[本层]符号表，再[查外层]，找不到则[报错]

· 符号表的另一种组织方式：将所有[块]的符号表放在一个[大数组]中，然后再引入一个[块表]来描述各[块]的符号表在大数组中的位置及相互关系；
一个[过程]可以看作是一个[块]。
display btab name tab
lastptr last name link



第八章 代码优化

8.1 流图

[基本块]是满足下列条件的[最大]的[连续]三地址指令序列。

① 控制流只能从基本块的[第一条指令]进入该块。

② 其除了基本块的[最后一条指令]，控制流在离开基本块之前之前不会[跳转]或者[停机]。

每个基本块由一组总是一起执行的指令组成

• 输入：三地址指令序列

输出：输入序列对应的基本块列表，其中每个指令恰好被分配给一基本块

方法：首先，确定指令序列中哪些指令是首指令，即某个基本块的第一条指令
① 指令序列的[第一个三地址指令]是一个首指令
② 任意一个条件或无条件转移指令的[目标指令]是一个首指令
③ 紧跟在一个条件或无条件转移指令之后的指令是一个首指令。然后，每个首指令对应的基本块包括了从它自己开始，直到[下一个首指令(不含)]或者[指令序列结尾]之间的指令。

• 流图 每个[结点]是一个[基本块]

从基本块B到基本块C之间有一条边当且仅当基本块C的第一条指令可能紧跟在B的最后一条指令之后执行。

8.2 优化的分类

• 机器无关优化：针对中间代码 • 机器相关优化：针对目标代码

• 局部代码优化：单个基本块范围内优化 • 全局代码优化：面向多个基本块优化

• 常用的优化方法：① 删除公共子表达式 ② 删除无用代码 ③ 代码移动

④ 强度削弱 ⑤ 删除归纳变量

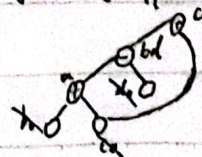
• 复制传播、常量合并、循环不变计算等概念。

8.3 基本块的优化

很多重要的[局部优化技术]首先把一[基本块]转换成[无环有向图/DAG]。

• 基本块中的每个语句s都对应一个内部结点N

eg. $a = b + c; b = a - d; c = b + c; d = a - d;$

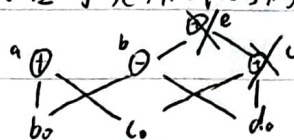


· 基于基本块的 DAG 删除无用代码

从一个 DAG 上删除所有没有附加活跃变量 (活跃变量是指其值可能会在以后被使用的变量) 的根结点 (即没有父结点的结点)。重复应用这样的处理过程就可以从 DAG 中消除所有对应于无用代码的结点。

eg. $a = b + c; \quad b = b - d;$

$c = c + d; \quad e = b + c;$



假设 a, b 是活跃变量

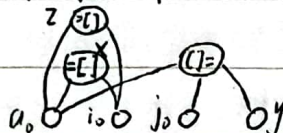
但 c 和 e 不是

· 数组元素赋值指令的表示

"[]" 结点有 3 个子结点, 无定值变量表

↳ $a, j, y \rightarrow a$ 的创建会杀死所有已建立的依赖于 a 的结点, 无法再获取定值变量

eg. $x = a[i];$
 $a[j] = y;$
 $z = a[j];$



· 跳转指令的表示 $\text{if } a < b \text{ goto } n$



· 根据基本块的 DAG 可以: ① 确定哪些变量的值在该基本块中赋值前被引用 ② 确定哪些语句计算的值可以在基本块外被引用

· 从 DAG 到基本块的重组: 对每个具有 [若干定值变量] 的节点, 构造一个 [地址语句] 来计算其中某个变量的值。

8.4 数据流分析

· 一组用来获取 [有关数据如何沿着程序执行路径流动] 的相关信息的技术

在每一种数据流分析应用中, 都会把每个 [程序点] 和一个 [数据流值] 关联起来

· 主要应用: 到达-定值分析, 活跃变量分析, 可用表达式分析

· 语句的数据流模式 $IN[s], OUT[s], f_s$

前向传播 $OUT[s] = f_s(IN[s])$ 逆向传播 $IN[s] = f_s(OUT[s]); \quad IN[s_m] = OUT[s_i]$

· 基本块上的数据流模式 $IN[B], OUT[B], f_B$

前向数据流 $f_B = f_{s_1} \cdot f_{s_2} \cdots f_{s_n} \cdot f_{s_{n+1}}$ 逆向数据流 $f_B = f_{s_1} \cdot f_{s_2} \cdots f_{s_m} \cdot f_{s_n}$

8.4.1 到达定值分析

概念: 定值 x , 定值 x 到达程序点 P

主要用途: ① 循环不变计算的检测 ② 常量传播 ③ 判断变量 x 在 P 上是否 [未经定值] 就 [被引用]。



· “生成”与“杀死”定值 $d: u = v + w$ 该语句“生成”了一个变量 u 的定值 d , 并“杀死”了程序中其它对 u 的定值

· 到达定值的传递函数 $f_d(x) = gen_d \cup x - kill_d$

$$f_B(x) = gen_B \cup (x - kill_B), \quad kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n, \quad gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup (gen_{n-2} - kill_{n-1} - kill_n) \cup \dots \cup (gen_1 - kill_2 - kill_3 - \dots - kill_n)$$

· 到达定值的数据流方程

$$OUT[ENTRY] = \phi \quad OUT[B] \text{ 初始化为 } \phi$$

$$OUT[B] = f_B(IN[B]) \quad (B \neq ENTRY), \quad f_B(x) = gen_B \cup x - kill_B$$

$$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P] \quad (B \neq ENTRY)$$

gen_B 和 $kill_B$ 的值可以直接从流图中计算出来, 因此在方程中作为已知量

· 引用-定值链 (ud 链): 一个列表, 对于变量的每一次引用, 到达该引用的所有定值都在该列表中。

8.4.2 活跃变量分析

· 活跃变量: 对于变量 x 和程序点 p , 如果流图中沿着从 p 开始的[某条路径]会引用变量 x 在 p 点的值, 则称变量 x 在点 p 是[活跃]的, 否则称变量 x 在点 p [不活跃]。

· 主要用途: 删除无用赋值 为基本块分配寄存器。

· 传递函数: $IN[B] = f_B(OUT[B]), \quad f_B = use_B \cup (x - def_B)$

· 数据流方程: $IN[EXIT] = \phi \quad IN[B] \text{ 初始化为 } \phi$

$$IN[B] = f_B(OUT[B]) \quad (B \neq EXIT), \quad f_B(x) = use_B \cup (x - def_B)$$

$$OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的后继}} IN[S] \quad (B \neq EXIT)$$

· 定值-引用链: 设变量 x 有一个定值 d , 该定值所有能够到达的引用 x 的集合称为 x 在 d 处的定值-引用链, 简称 d - u 链。

8.4.3 可用表达式分析

· 如果从流图的[首节点]到达程序点 p 的[每条路径]都对表达式 $x \text{ op } y$ 进行计算, 并且从最后一个这样的计算到点 p 之间[没有再次对 x 或 y 定值]那么表达式 $x \text{ op } y$ 在点 p 是[可用的]



· 主要用途：消除全局公共子表达式，进行复制传播

· 传递函数： $f_B(x) = e-gen_B \cup (x - e-kill_B)$

$e-gen_B$ 计算：初始 $e-gen_B = \phi$ ，顺序扫描基本块中每个语句 $z = x \text{ op } y$

把 $x \text{ op } y$ 加入 $e-gen_B$ ，从 $e-gen_B$ 中删除和 z 相关表达式

$e-kill_B$ 计算：初始化 $e-kill_B = \phi$ ，顺序扫描基本块中每个语句 $z = x \text{ op } y$

从 $e-kill_B$ 中删除表达式 $x \text{ op } y$ ，把所有和 z 相关表达式加入 $e-kill_B$ 中

· 数据流方程： $OUT[ENTRY] = \phi$ $OUT[B]$ 初始化为 U

$$OUT[B] = f_B(IN[B]) \quad (B \neq ENTRY)$$

$$f_B(x) = e-gen_B \cup (x - e-kill_B)$$

$$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱}} OUT[P] \quad (B \neq ENTRY)$$

8.5 流图中的循环

· 支配结点：如果从流图的[入口]结点到结点 n 的[每条路径]都经过结点 d 则称结点 d 支配结点 n ，记为 $d \text{ dom } n$

数据方程： $OUT[ENTRY] = \{ENTRY\}$

$$OUT[B] = f_B(IN[B]) \quad (B \neq ENTRY)$$

$$f_B(x) = x \cup \{B\}$$

n, d 可以是同一个结点

$$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱}} OUT[P] \quad (B \neq ENTRY)$$

· 如果存在从结点 n 到 d 的有向边 $n \rightarrow d$ ，且 $d \text{ dom } n$ ，那么这条边称为[回边]

· 自然循环：一种适合优化的循环，满足①有唯一的入口结点，称为首结点②循环中至少有一条返回首结点的路径。

· 自然循环的识别：给定一个回边 $n \rightarrow d$ ，该[回边的自然循环]为： d ，以及所有可以[不经过 d 而到达 n 的结点]。 d 为该循环的首结点。

· 自然循环的重要性质：

①如果两个自然循环的[首结点不相同]，则这两个循环要么[互不相交]，要么一个[完全包含(嵌入)在另外一个里面]②如果两个循环具有[相同的首结点]，那么很难说哪个是最内循环。此时把两个循环合并。

最内循环：不包含其它循环的循环



- 数据流分析的主要应用：①到达定值 $\downarrow$ 正向 ②活跃变量 $\downarrow$ 逆向
③可用表达式 $\downarrow$ 正向 ④支配结点 $\downarrow$ 正向

8.6 全局优化

删除全局公共子表达式，删除复制语句，代码移动

作用于递归变量的强度减弱，删除递归变量

8.6.1 删除全局公共子表达式

· [可用表达式]的数据流问题可以帮助我们确定位于流图中 p 点的表达式是否为[全局公共子表达式]

8.6.2 删除复制语句

· 对于复制语句 $s: x=y$ ，如果在 x 的所有引用者可以用对 y 的引用替代对 x 的引用（复制传播），那么可以删除复制语句 $x=y$

· 在 x 的引用点 u 用 y 代替 x （复制传播）的条件

复制语句 $s: x=y$ 在 u 点“可用”

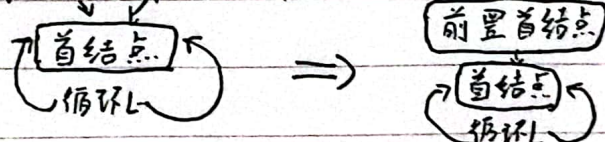
算法详见 PPT P122

· 通过 du 链和有对 x 的引用块 B 的 $ZN[B]$ 的对比来判断可删除的复制语句

8.6.3 代码移动

循环不变计算检测算法 详见 PPT P124

代码外提：



循环不变计算语句 $s: x=y+z$ 移动条件 ① s 所在的基本块是循环[所有出口结点]（有后继结点在循环外的结点）的[支配结点] ② 循环中没有其它语句对 x 赋值 ③ 循环中对 x 的引用仅由 s 到达。

代码移动算法 输入循环 L ， du 链，支配结点信息等进行改。详见 PPT P129

8.6.4 作用于归纳变量的强度削弱

- 循环 L 的[基本归纳变量]：循环 L 中的变量 i 只有形如 $i=i+c$ 的赋值
- 如果 $j=c.i+d$ ，其中 i 是基本归纳变量， c 和 d 是常量，则 j 也是一个归纳变量，称 j 属于 i 族。

· 归纳变量检测算法。输入带有[循环不变计算]信息和[到达定值]信息的循环 L

详见 PPT P134



· 作用于归纳变量的强度削弱算法 详见PPT P136

输入带有[到达点值信息]和已计算出的[归纳变量族]的循环L

8.6.5 归纳变量的删除

· 对于在[强度削弱]算法中引入的复制语句 $j=t$, 如果在[归纳变量 j]的所有引用点, 都可以^用对 t 的引用[代替]对 j 的引用, 并且 j 在循环出口处不活跃, 则可以删除复制语句 $j=t$.

· 强度削弱后, 有些归纳变量的作用[只是用于测试]. 如果可以用对[其它归纳变量]的测试[代替]对这种归纳变量的测试, 那么可以删除这种归纳变量.



第九章 代码生成

9.1 代码生成器的主要任务.

指令选择: 选选择适当的[目标机指令]来实现[中间表示 (IR) 语句]

寄存器分配和指派: 把哪个值放在哪个寄存器中.

指令排序: 按照什么顺序来安排指令的执行.

9.2 一个简单的目标机模型

三地址机器模型: 加载^等保存、运算、跳转操作; 内存按[字节]寻址;
n个通用寄存器 $R_0, R_1, \dots, R_{n-1}$; 假设所有运算分量都是[整数]; 指令之间可能
有一个[标号].

主要指令: 加载 LD, 保存 ST, 运算 OP, 无条件跳转 BR, 条件跳转 Bcond

寻址模式: 变量名 a, 常数 c, 寄存器 r: $a(r), c(r), *r, *(c(r)), \#c$

9.3 指令选择

运算语句, 数组寻址语句, 指针存取语句, 条件跳转, 过程调用和返回

$a(r)$

$c(r), c \neq 0$

静态/栈式内存分配

三地址语句

call callee

return

目标代码(静态方式)

ST callee, static Area where +20
BR callee, code Area

BR *callee, static Area

目标代码(栈式)

ADD SP, SP, #callee, record size
ST 0(SP), where +16
BR callee, code Area

被调用过程: BR *0(SP)

调用过程: SUB SP, SP, #callee, record size

9.4 寄存器的选择

· 函数 $getReg(I)$ 为 x, y, z 选择寄存器 R_x, R_y, R_z

· 寄存器描述符: 记录每个[寄存器]当前存放的是哪些变量值.

地址描述符: ①记录运行时每个[名字]的当前值放在哪个或哪些位置

②该位置可能是寄存器、栈单元、内存地址或是它们的某个集合 ③这些信息可以存放在该变量名对应的[符号表]条目中.

· 基本块的收尾处理

忘记原有在基本块内部使用的临时变量

存放出口处可能[活跃]的变量 x " ST x, R "



· 管理寄存器和地址描述符：当代码生成算法生成加载保存和其它指令时，它必须同时更新寄存器和地址描述符。

"LD R, X", "OP R₁, R₂, R₃", "ST X, R", $X=Y$ 分别有 3, 4, 1, 5 步操作。

· 寄存器选择函数 `getReg` 详见 PPT P47 流程图

· 计算寄存器 R 的费用 详见 PPT P48 流程图

· 寄存器 R₁ 的选择与 R₂ 相似不过因为 X 是一个正在被计算的新值，增加两个优化 ① 只存放了 X 的值的寄存器对 R₁ 来说总是可接受的 ② 如果 Y 在指令 1 后不再使用，且 R₂ 仅仅保存了 Y 值，那么 R₂ 同时可用作 R₁。R₂ 同理

· 当 1 是复制指令 $X=Y$ 时，选择好 R₂ 后，令 $R_1=R_2$

9.5 窥孔优化

· 相当于维持一个滑动窗口，如果满足某些条件，就用更快或更短的指令来替换窗口中的指令序列。

9.6.1 冗余指令的删除

· 消除冗余的加载和保存指令 (有标号不能删除)

· 消除不可达代码 (eg. 紧跟在无条件跳转之后的不带标号的指令)

9.6.2 控制流优化

在代码中出现跳转到跳转指令的指令时，某些条件下可以使用一个跳转指令替代。

9.6.3 代数优化

· 代数恒等式 $x=x+0$; $x=x \times 1$;

· 强度削弱 乘 2 的幂 $\rightarrow$ 移位 除法 $\rightarrow$ 乘法

9.6.4 机器特有指令的使用 (特殊指令的使用)

· 充分利用目标系统的某些高效的特殊指令来提高代码效率

eg. INC 替代加 1.

